

표준입출력과 파일 디스크립터 분석 보고서

- 학과: 인공지능
- 학번: 20233136
- 이름: 나현우

주제: 표준입출력과 파일 디스크립터 분석

1. 이론 설명

1.1 파일 디스크립터란 무엇인가

파일 디스크립터(File Descriptor)는 POSIX 계열 운영체제에서 열려 있는 입출력 대상을 가리키는 정수값이다. 프로그램은 파일, 터미널, 파이프, 소켓을 직접 다루는 것이 아니라 `read`, `write`, `close` 같은 시스템 콜에 파일 디스크립터 번호를 전달하여 입출력을 수행한다.

파일 디스크립터가 정수로 표현되는 이유는 프로세스마다 커널이 관리하는 파일 디스크립터 테이블이 있고, 파일 디스크립터가 그 테이블의 인덱스 역할을 하기 때문이다. 예를 들어 어떤 프로세스에서 3이라는 파일 디스크립터가 열린 파일을 가리키고 있다면, 커널은 해당 프로세스의 파일 디스크립터 테이블에서 3번 항목을 찾아 실제 열린 파일 객체, 현재 파일 위치, 접근 모드 등을 확인한다.

POSIX 계열 운영체제에서 파일 디스크립터는 다음과 같은 역할을 한다.

- 프로세스가 열어 둔 입출력 대상의 참조 번호 역할을 한다.
- 일반 파일, 표준입출력, 파이프, 소켓, 장치 파일을 거의 같은 방식으로 다루게 한다.
- Shell 리다이렉션과 파이프가 동작할 수 있는 공통 기반을 제공한다.
- `dup`, `dup2`, `pipe`, `open`, `close`, `read`, `write` 같은 시스템 콜의 핵심 인자로 사용된다.

Windows 운영체제는 POSIX의 파일 디스크립터 대신 `HANDLE`이라는 커널 객체 핸들을 주로 사용한다. 다만 Visual C++ 런타임이나 MinGW 같은 C/C++ 실행 환경은 `_open`, `_read`, `_write`, `_fileno` 처럼 POSIX와 비슷한 함수와 작은 정수 기반 파일 디스크립터 계층을 제공한다. 즉 Windows 내부의 기본 모델은 `HANDLE` 이고, C/C++ 런타임이 이를 파일 디스크립터처럼 사용할 수 있게 감싸는 구조라고 볼 수 있다.

`fileno`는 C의 `FILE*` 스트림이 내부적으로 사용하는 파일 디스크립터 번호를 얻는 함수이다. POSIX에서는 `fileno(stdout)` 처럼 사용하고, Windows의 MSVC 계열 C 런타임에서는 보통 `_fileno(stdout)` 처럼 앞에 밑줄이 붙은 함수를 사용한다. 예를 들어 `_fileno(stdout)`의 결과는 일반적으로 표준 출력 파일 디스크립터인 1번이다.

Windows에서 C/C++ 런타임은 `FILE*`, 파일 디스크립터, Windows `HANDLE` 사이를 이어주는 중간 계층 역할을 한다. 구조를 단순화하면 다음과 같다.

```

C++ std::cout
    ↓
C++ stream buffer
    ↓
C stdout / FILE*
    ↓
C 런타임 파일 디스크립터 1번
    ↓
Windows HANDLE
    ↓
콘솔 / 파일 / 파이프

```

예를 들어 `_write(1, buffer, size)`가 호출되면 C 런타임은 파일 디스크립터 1번이 어떤 Windows `HANDLE`에 연결되어 있는지 찾고, 최종적으로 Windows API를 통해 해당 핸들에 데이터를 쓴다. 반대로 `_get_osfhandle(fd)`는 C 런타임의 파일 디스크립터를 Windows `HANDLE`로 바꿔 주고, `_open_osfhandle(handle, flags)`는 Windows `HANDLE`를 C 런타임 파일 디스크립터로 등록할 때 사용된다.

1.2 표준입출력의 파일 디스크립터

POSIX 시스템에서 프로그램이 시작될 때 기본적으로 열려 있는 표준 스트림은 다음과 같다.

이름	파일 디스크립터	의미
<code>stdin</code>	0	표준 입력. 기본적으로 키보드나 파이프, 입력 파일과 연결된다.
<code>stdout</code>	1	표준 출력. 기본적으로 터미널 화면이나 출력 파일과 연결된다.
<code>stderr</code>	2	표준 에러. 기본적으로 터미널 화면이나 에러 출력 파일과 연결된다.

이 세 스트림은 프로그램 코드가 직접 열지 않아도 프로그램 시작 시점에 이미 준비되어 있다. 일반적으로 사용자가 Shell에서 프로그램을 실행하면 Shell이 자식 프로세스를 만들기 전에 표준입출력 파일 디스크립터를 설정하고, 자식 프로세스는 이를 상속받는다. 운영체제는 파일 디스크립터 테이블과 상속 기능을 제공하고, Shell은 리다이렉션이나 파이프 명령에 맞게 0, 1, 2번 디스크립터를 터미널, 파일, 파이프 등으로 연결한다.

예를 들어 다음 명령에서 Shell은 프로그램 실행 전에 `out.txt` 를 열고 그 파일을 파일 디스크립터 1번, 즉 `stdout` 에 연결한다.

```
./test > out.txt
```

프로그램 입장에서는 여전히 `stdout` 에 출력할 뿐이지만, Shell이 미리 연결 대상을 바꾸었기 때문에 출력은 화면이 아니라 파일로 이동한다.

1.3 C/C++ IO와 파일 디스크립터

C++ 표준입출력 객체와 C 표준 스트림, 운영체제 파일 디스크립터는 보통 다음 관계로 연결된다.

C++ 객체	C 표준 스트림	POSIX 파일 디스크립터
<code>std::cin</code>	<code>stdin</code>	0
<code>std::cout</code>	<code>stdout</code>	1
<code>std::cerr</code>	<code>stderr</code>	2

`std::cin`, `std::cout`, `std::cerr` 는 C++ 표준 라이브러리의 `iostream` 객체이다. 이 객체들은 직접 운영체제 커널을 호출한다기보다, 내부의 `stream buffer`와 C/C++ 런타임을 거쳐 최종적으로 운영체제의 입출력 기능에 도달한다. 기본 설정에서는

`std::ios::sync_with_stdio(true)` 상태이므로 C++ `iostream`과 C의 `stdio` 스트림이 동기화된다.

`printf` 와 `cout` 은 출력한다는 점에서는 비슷하지만 내부 구조와 사용 방식이 다르다.

구분	<code>printf</code>	<code>std::cout</code>
소속	C 표준 라이브러리	C++ 표준 라이브러리
출력 방식	형식 문자열 기반	<code><<</code> 연산자 기반
타입 처리	형식 지정자와 인자의 일치가 중요	연산자 오버로딩으로 타입별 출력 처리
버퍼	<code>FILE*</code> 기반 <code>stdio</code> 버퍼	<code>std::streambuf</code> 기반 버퍼

최종 출력	런타임을 거쳐 OS write 계열 기능 사용	런타임을 거쳐 OS write 계열 기능 사용
-------	---------------------------	---------------------------

결론적으로 C++ 코드는 `std::cout` 에 출력하는 것처럼 보이지만, 운영체제 수준에서는 결국 파일 디스크립터 1번 또는 그에 대응하는 운영체제 핸들로 바이트가 전달된다.

2. 표준입출력 프로그램 작성

다음 프로그램은 `stdin` 에서 한 줄씩 입력을 읽고, 읽은 내용을 그대로 `stdout` 에 출력한다.

```
#include <iostream>
#include <string>

int main() {
    std::string line;

    while (std::getline(std::cin, line)) {
        std::cout << line << std::endl;
    }
}
```

이 프로그램은 입력 대상이 키보드인지, 파일인지, 파이프인지 직접 알 필요가 없다.

`std::cin` 이 연결된 표준 입력 스트림에서 읽을 뿐이다. Shell이 `stdin` 의 연결 대상을 바꾸면 같은 코드가 키보드 입력도 읽고 파일 내용도 읽고 파이프에서 전달된 데이터도 읽을 수 있다.

3. `stdout` `stderr` 리다이렉션 실험

3.1 실험 프로그램

다음 프로그램을 `test.cpp` 로 저장하고 실행 파일 이름이 `test` 가 되도록 빌드한다.

```
#include <iostream>

int main() {
    std::cout << "This is stdout message" << std::endl;
    std::cerr << "This is stderr message" << std::endl;
}
```

빌드 예시는 다음과 같다.

```
g++ test.cpp -o test
```

Windows에서 MinGW를 사용하는 경우 실행 파일 이름은 보통 `test.exe` 가 된다.

3.2 실험 1: 기본 실행

```
./test
```

실행 결과는 터미널에 다음과 같이 출력된다.

```
This is stdout message
This is stderr message
```

아무 리다이렉션을 하지 않았으므로 `stdout` 과 `stderr` 가 모두 터미널에 연결되어 있다. 따라서 두 출력이 모두 화면에 보인다.

3.3 실험 2: stdout 리다이렉션

```
./test > out.txt
```

실행 결과는 다음과 같이 나뉜다.

위치	내용
터미널	<code>This is stderr message</code>
<code>out.txt</code>	<code>This is stdout message</code>

`>` 는 기본적으로 파일 디스크립터 1번, 즉 `stdout` 을 파일로 리다이렉션한다. 따라서 `std::cout` 출력은 `out.txt` 로 이동한다. 반면 `std::cerr` 는 파일 디스크립터 2번인 `stderr` 를 사용하므로 여전히 터미널에 출력된다.

3.4 실험 3: stderr 리다이렉션

```
./test 2> err.txt
```

실행 결과는 다음과 같이 나뉜다.

위치	내용
터미널	<code>This is stdout message</code>

err.txt	This is stderr message
---------	------------------------

2> 에서 숫자 2 는 파일 디스크립터 2번을 의미한다. 즉 `stderr` 를 `err.txt` 로 연결하라는 뜻이다. 이때 `stdout` 은 변경하지 않았으므로 `std::cout` 출력은 터미널에 그대로 표시된다.

4. stdout과 stderr 분리

다음 명령은 `'stdout'`과 `'stderr'`를 서로 다른 파일로 분리한다.

```
./test > out.txt 2> err.txt
```

실행 후 각 파일의 내용은 다음과 같다.

파일	내용	연결된 파일 디스크립터
out.txt	This is stdout message	1
err.txt	This is stderr message	2

이 실험은 표준 출력과 표준 에러가 서로 독립적인 스트림이라는 사실을 보여준다. 둘 다 기본적으로 터미널에 보이기 때문에 하나처럼 느껴질 수 있지만, 파일 디스크립터 관점에서는 1번과 2번으로 분리되어 있다.

5. stdout과 stderr를 하나로 합치기

다음 명령은 `stdout` 과 `stderr` 를 모두 하나의 파일로 보낸다.

```
./test > all.txt 2>&1
```

실행 후 `all.txt` 에는 다음 내용이 기록된다.

```
This is stdout message
This is stderr message
```

2>&1 은 파일 디스크립터 2번을 파일 디스크립터 1번이 현재 가리키는 곳으로 복제하라는 의미이다. 이 명령은 왼쪽에서 오른쪽으로 해석된다.

1. `> all.txt` 에 의해 `stdout` , 즉 파일 디스크립터 1번이 `all.txt` 로 연결된다.
2. `2>&1` 에 의해 `stderr` , 즉 파일 디스크립터 2번도 현재 1번이 가리키는 `all.txt` 로 연결된다.

따라서 `std::cout` 과 `std::cerr` 출력이 모두 같은 파일에 저장된다. 파일 디스크립터 관점에서 보면 1번과 2번이 같은 출력 대상을 가리키도록 설정된 것이다.

6. stdin 리다이렉션 실험

6.1 실험 프로그램

다음 프로그램을 `read_input.cpp` 로 저장하고 실행 파일 이름이 `read_input` 이 되도록 빌드한다.

```
#include <iostream>
#include <string>

int main() {
    std::string line;

    std::cout << "Reading from STDIN..." << std::endl;

    while (std::getline(std::cin, line)) {
        std::cout << "Input : " << line << std::endl;
    }
}
```

빌드 예시는 다음과 같다.

```
g++ read_input.cpp -o read_input
```

6.2 실험 1: 키보드 입력

```
./read_input
```

입력 예시는 다음과 같다.

```
apple
banana
orange
```

실행 결과는 다음과 같다.

```
Reading from STDIN...
Input : apple
Input : banana
Input : orange
```

키보드에서 입력한 문자열이 `std::cin` 을 통해 프로그램으로 들어오고, 프로그램은 각 줄 앞에 `Input :` 을 붙여 `std::cout` 으로 출력한다. 입력 종료는 Unix/Linux/macOS에서는 `Ctrl+D` , Windows 콘솔에서는 보통 `Ctrl+Z` 후 Enter로 전달할 수 있다.

`Ctrl+C` 는 입력 종료를 의미하는 EOF가 아니라 실행 중인 프로그램에 중단 요청을 보내는 키이다. Unix/Linux/macOS에서는 보통 `SIGINT` 신호가 전달되고, Windows 콘솔에서도 비슷한 콘솔 제어 이벤트가 전달되어 프로그램이 중단된다. 따라서 `std::getline` 반복문을 정상적으로 끝내려면 EOF를 보내는 `Ctrl+D` 또는 Windows의 `Ctrl+Z` 후 Enter를 사용하고, 프로그램을 강제로 멈추고 싶을 때 `Ctrl+C` 를 사용한다.

6.3 실험 2: 파일로 stdin 대체

먼저 `input.txt` 를 다음 내용으로 만든다.

```
apple
banana
orange
```

그 다음 다음 명령을 실행한다.

```
./read_input < input.txt
```

실행 결과는 다음과 같다.

```
Reading from STDIN...
Input : apple
Input : banana
Input : orange
```

프로그램 코드를 수정하지 않았는데 파일을 읽을 수 있는 이유는 Shell이 프로그램 실행 전에 파일 디스크립터 0번, 즉 `stdin` 을 키보드 대신 `input.txt` 에 연결했기 때문이다. 프로그램은 여전히 `std::cin` 에서 읽지만, `std::cin` 이 연결된 실제 입력 대상이 파일로 바뀐 것이다.

파일 디스크립터 관점에서는 다음과 같이 볼 수 있다.

실행 방식	파일 디스크립터 0번의 연결 대상
<code>./read_input</code>	터미널 입력
<code>./read_input < input>txt</code>	<code>input.txt</code>

7. 파이프 실험

Unix/Linux/macOS에서는 다음 명령을 실행할 수 있다.

```
cat input.txt | ./read_input
```

Windows에서는 다음 명령을 사용할 수 있다.

```
type input.txt | read_input.exe
```

간단한 입력은 다음처럼 `echo`로 전달할 수도 있다.

```
echo "hello world" | ./read_input
```

`echo "hello world" | ./read_input`의 실행 결과는 다음과 같다.

```
Reading from STDIN...
Input : hello world
```

파이프의 동작 원리는 두 프로세스 사이에 커널이 관리하는 임시 데이터 통로를 만드는 것이다. Shell은 왼쪽 명령의 `stdout`을 파이프의 쓰기 쪽에 연결하고, 오른쪽 명령의 `stdin`을 파이프의 읽기 쪽에 연결한다.

여기서 `cat input.txt`는 파일을 새로 작성하는 명령이 아니라, `input.txt`를 읽어서 그 내용을 `stdout`으로 출력하는 명령이다. 파이프를 사용하지 않으면 `cat`의 `stdout`은 터미널에 연결되어 파일 내용이 화면에 보인다. 파이프를 사용하면 `cat`의 `stdout`이 터미널이 아니라 파이프의 쓰기 쪽에 연결되므로, `cat`이 출력한 내용이 파이프 안으로 들어간다.

파일 디스크립터 관점에서 `cat input.txt | ./read_input`은 다음과 같이 동작한다.

프로세스	파일 디스크립터	연결 대상
<code>cat input.txt</code>	1 <code>stdout</code>	파이프의 쓰기 끝

<code>./read_input</code>	<code>0 stdin</code>	파이프의 읽기 끝
---------------------------	----------------------	-----------

따라서 `read_input` 프로그램은 파일을 직접 열지 않았지만 `std::cin`으로 데이터를 읽을 수 있다. 그 데이터는 키보드가 아니라 파이프를 통해 전달된 이전 명령의 출력이다.

`cat`이 `input.txt`의 끝까지 읽으면 더 이상 `stdout`에 쓸 데이터가 없으므로 `cat` 프로세스가 종료된다. 그러면 `cat`이 사용하던 파이프의 쓰기 쪽도 닫힌다. `read_input`은 파이프에 남아 있는 데이터를 모두 읽은 뒤 쓰기 쪽이 닫힌 것을 보고 EOF를 만나며, `std::getline` 반복문을 종료한다.

`./read_input < input.txt`와 `cat input.txt | ./read_input`은 최종 데이터의 원본이 모두 `input.txt`라는 점에서는 같다. 그러나 `read_input`의 `stdin`에 직접 연결된 대상은 다르다.

명령	데이터 원본	<code>read_input</code> 의 <code>stdin</code> 연결 대상	중간 프로세스
<code>./read_input < input.txt</code>	<code>input.txt</code>	파일 <code>input.txt</code>	없음
<code>cat input.txt</code>	<code>input.txt</code>	파이프의 읽기 끝	<code>cat</code>

즉 `< input.txt`는 `read_input`이 표준 입력으로 파일을 직접 읽는 구조이고, `cat input.txt | ./read_input`은 `cat`이 파일을 읽어 파이프에 써 넣고 `read_input`이 그 파이프에서 읽는 구조이다.

8. 분석 및 결론

8.1 프로그램은 파일을 읽는 것이 아니라 무엇을 읽는가?

프로그램은 특정 파일 자체를 직접 읽는 것이 아니라, 자신에게 연결된 입력 스트림을 읽는다. POSIX 관점에서는 파일 디스크립터 0번, 즉 `stdin`으로부터 바이트 스트림을 읽는다. 이 스트림의 실제 연결 대상은 상황에 따라 터미널, 일반 파일, 파이프, 소켓 등이 될 수 있다.

따라서 다음 세 명령은 코드가 같아도 입력 출처만 달라진다.

```
./read_input
./read_input < input.txt
cat input.txt | ./read_input
```

프로그램은 모두 `std::cin`에서 읽지만, Shell이 파일 디스크립터 0번의 연결 대상을 다르게 설정한다. 특히 `./read_input < input.txt`와 `cat input.txt | ./read_input`은 데이터의 원본

이 둘 다 `input.txt` 일 수 있지만, 전자는 `stdin` 이 파일에 직접 연결되고 후자는 `stdin` 이 파이프에 연결된다는 차이가 있다.

8.2 `stdin` `stdout` `stderr` 는 무엇인가?

`stdin`, `stdout`, `stderr` 는 프로그램이 시작할 때 기본적으로 제공되는 표준 스트림이다.

표준 스트림	방향	일반적인 용도	POSIX 파일 디스크립터
<code>stdin</code>	입력	사용자 입력, 파일 입력, 파이프 입력	0
<code>stdout</code>	출력	정상 결과 출력	1
<code>stderr</code>	출력	오류 메시지, 진단 메시지 출력	2

`stdout` 과 `stderr` 가 분리되어 있기 때문에 정상 결과는 파일로 저장하면서 오류 메시지는 화면에서 확인하거나, 반대로 오류만 별도 파일로 저장하는 일이 가능하다.

8.3 리다이렉션과 파이프 명령이 가능한 이유

리다이렉션과 파이프가 가능한 이유는 프로그램이 구체적인 입력 및 출력 장치를 직접 고정하지 않고, 표준 스트림과 파일 디스크립터를 통해 입출력을 수행하기 때문이다. Shell은 프로그램을 실행하기 전에 파일 디스크립터 0, 1, 2번이 가리키는 대상을 바꿀 수 있다.

예를 들어 다음 명령은 파일 디스크립터 1번과 2번을 모두 같은 파일로 연결한다.

```
./test > all.txt 2>&1
```

또 다음 명령은 왼쪽 프로그램의 `stdout` 과 오른쪽 프로그램의 `stdin` 사이에 파이프를 연결한다.

```
cat input.txt | ./read_input
```

이처럼 프로그램은 `std::cin`, `std::cout`, `std::cerr` 만 사용하지만, Shell과 운영체제가 파일 디스크립터의 연결 대상을 조정하기 때문에 파일 입력, 파일 출력, 에러 분리, 출력 병합, 파이프 연결이 모두 가능하다.

Appendix. 작성한 코드

A. `stdin`을 `stdout`으로 그대로 출력하는 프로그램

```

#include <iostream>
#include <string>

int main() {
    std::string line;

    while (std::getline(std::cin, line)) {
        std::cout << line << std::endl;
    }
}

```

B. stdout과 stderr를 출력하는 프로그램

```

#include <iostream>

int main() {
    std::cout << "This is stdout message" << std::endl;
    std::cerr << "This is stderr message" << std::endl;
}

```

C. stdin 입력을 표시하는 프로그램

```

#include <iostream>
#include <string>

int main() {
    std::string line;

    std::cout << "Reading from STDIN..." << std::endl;

    while (std::getline(std::cin, line)) {
        std::cout << "Input : " << line << std::endl;
    }
}

```